

Software/ML Technical Task Phase Report

Submitted by

Name: M. Arya

Registration Number: 240961162

Branch: B.Tech – Cyber Physical Systems

Institution: Manipal Institute of Technology

Repository: https://github.com/Arya42-cmd/Synergy_TP

Submission Date: 28 June 2026

Abstract

This report summarizes the theoretical concepts learned during the Software/ML Technical Task Phase. It discusses repository management using Git and GitHub, Python virtual environments, dependency management, Linux command-line basics, Python programming concepts, CSV processing, manual parsing and Pandas, data cleaning techniques, dataset validation, and data visualization with Matplotlib. The report focuses on the reasoning behind these concepts and the observations obtained from Tasks 1–5 rather than implementation details or source code. It also explains how raw data is transformed into a validated dataset and meaningful visualizations through a structured workflow.

1. Introduction

From the start, this phase was about digging in and getting my hands dirty—not just reading about best practices, but actually implementing them. Rather than sticking to theory, every task called for a working solution, from the smallest file read to the last PNG chart. The first thing on my plate was organizing: configuring Git and GitHub, setting up a virtual Python

environment, and keeping everything in a logical folder structure. These small rituals really do set the tone for clean, sustainable development.

Early tasks focused on understanding and manipulating CSV files holding student submission data. At first, I handled file parsing manually using basic Python tools, so I could see first-hand how structured data is stored and read. Pretty soon, though, I switched to Pandas. That shift was like moving from hand tools to power tools—suddenly, filtering and summarizing the data got so much faster and more reliable.

Then came the grunt work: cleaning up input data. The dataset had classic problems—duplicate entries, missing numbers, inconsistent category names and units, and a handful of rogue or nonsensical values. Fixing these wasn't glamorous, but it was necessary. Only once the dataset was trustworthy did I move on to analyzing trends and visualizing information with Matplotlib. The visualizations transformed those cleaned numbers into stories—a chart showing how attendance connects to scores, another breaking down performance across different domains, and a snapshot of students' submission rates. Along the way, I kept everything modular, tidy, and well documented.

In short, the project created a pipeline that combined foundational engineering habits with real-world programming and data skills—exactly the sort of routine work that powers actual software or ML projects.

2. Development Environment and Version Control

Reliable software starts with a solid workspace. I created the "Synergy_TP" GitHub repository to hold everything, splitting tasks into folders and making sure each piece of code and its documentation were easy to find. Using Git wasn't just a formality—it meant every change, from a bug fix to a major refactor, was documented and reversible. Each commit got a clear message, and everything was regularly pushed so the project's history stayed transparent and safe.

The repository followed a structured layout consisting of separate folders for each task together with README.md, requirements.txt and .gitignore files. Linux command-line utilities such as pwd, ls, cd, mkdir, touch, cat, find and chmod were used for directory navigation, file creation, inspection, searching and permission management. These commands support efficient project organization and are commonly used in software development workflows.

A key step was building a Python virtual environment for the project's dependencies. This isolated the work from the rest of my system and kept every library—no matter how specific or oddball—tied to the project, avoiding version conflicts down the road. The requirements.txt file collected all dependencies, making setup a breeze for anyone else using the repository; a single pip install command could recreate my working environment on another machine.

The .gitignore file also played a quiet but crucial role. It kept non-essential junk—local environment folders, temporary directories, editor configs—out of the repository. Looking back, these early organizational choices made the rest of the project faster, less stressful, and much easier to maintain.

All these things—Git, GitHub, virtual environments—weren't luxuries. They're backbone habits for any technical team, and they paid off immediately during each twist of the project.

3. Python Programming and File Handling

Why Python? Because it just makes sense for this kind of work. It's accessible, yet powerful, with a ton of libraries that make data and file handling straightforward. The code was built around functional breakdown: each core job—like reading a file, cleaning a particular column, generating a summary, or drawing a chart—got its own function. This modular style didn't just keep the code readable, it made bugs less intimidating and new features less risky to add.

Robustness mattered, too. I built in exception handling, so a missing or corrupted CSV file didn't crash the program; instead, it gave a helpful message and let me debug without headaches. Mapping and type conversion were routine: whenever data appeared in slightly different formats (like “ml” vs. “machine learning”), dictionaries handled the translation to a standard form. Same idea for handling numbers written as words, or converting measurements into unified units.

File operations went beyond just opening CSVs. The project saved results as JSON summaries, produced Markdown reports, and exported charts as PNGs. Python's built-in methods, along with just a handful of libraries, handled all the heavy lifting—from reading line by line to compiling the cleaned results for future analysis or presentation.

This phase genuinely strengthened my understanding of how core Python features, paired with good structure and error handling, can take real-world, messy data and turn it into something useful.

4. CSV Parsing and Pandas

CSV files are everywhere, and I wanted to understand them inside and out. I started by writing a basic parser, opening the file, splitting lines by commas, stripping out whitespace, and assembling dictionaries keyed by column names. Right away, I saw the quirks: data arrives as strings, so before I could do math, I had to explicitly convert scores and attendance values to numbers. And if the data was written out in words—say, “two” or “nine”—I mapped those to digits as well.

Manual parsing, though, gets old fast. Handling missing or malformed data means a lot of extra code, and mistakes become easy to make and hard to catch. That's where Pandas entered the picture. With one function, `read_csv()`, I loaded the data into a `DataFrame`—a

robust, highly efficient structure that's pretty much the backbone of modern data science work. Filtering, summarizing, and aggregating all became concise and readable.

Using Pandas didn't just save time. It shifted my mindset from managing the nitty-gritty of data formatting to actually analyzing the data. I could quickly grab averages, totals, or filtered groups with a single line. Comparing both approaches made the lesson clear: understanding the "manual" way builds insight, but for anything beyond toy datasets, Pandas is the professional's tool of choice. It's reliable, battle-tested, and loaded with power features.

5. Data Cleaning and Validation

No one hands you a perfect dataset. Fixing dirty data is a huge chunk of any analysis, and this project was no exception. First, I checked for and removed duplicate entries—because one record slipping through twice can throw off every summary. Then, I standardized categorical labels. If “ml,” “ML,” or “MACHINE LEARNING” all referred to the same thing, they all got converted into a single, predictable format.

Type inconsistencies were everywhere: percentages included symbols, heights toggled between centimeters and meters, weights appeared with various units, and some numbers were spelled out. I created consistent rules for converting every one to its proper numeric representation, ensuring I could run calculations without special-casing anything.

Handling missing values required care. Instead of just deleting records with blanks, I filled in gaps for numeric data with the median value from each column—since the median resists the influence of extreme outliers. Attendance values were brought back into the valid range, eliminating impossible or incorrect percentages that would mess up later analysis.

Once the cleaning was done, I validated the data. I checked for unique IDs, valid column types, properly constrained values, and consistent standards everywhere—but mostly, I verified that nothing had slipped through the cracks during transformation. Skipping this step would

have meant risking subtle errors later. By validating, I made sure the dataset was truly ready for its next phase.

6. Data Visualization

Three visualizations were generated to communicate different characteristics of the cleaned dataset. The first bar chart compares average scores across domains, the scatter plot illustrates the relationship between attendance and scores, and the final bar chart summarizes submission status. Appropriate titles, axis labels and saved PNG outputs improve readability, reproducibility and documentation.

After all the cleansing and validation, the data was primed for visual exploration. Numbers on their own can be abstract and hard to interpret quickly, so I used Matplotlib to bring key findings to life. Three visualizations captured different aspects of the dataset.

I started with a bar chart, laying out average scores by domain to show at a glance which groups performed best. This provided immediate, business-relevant insights that a table of numbers would have buried. Next, I created a scatter plot connecting attendance rates to scores—each point a student, mapping attendance on the x-axis against performance on the y-axis. This kind of chart is a direct way to spot trends, clusters, or outliers. Even with a modest dataset, the visual made patterns pop.

Finally, I drew a bar chart summarizing which students completed their submissions and which didn't—a straightforward but effective way to measure participation and compliance.

For each visualization, I made sure to include descriptive titles and clear axis labels. Instead of popping them up on-screen, I saved the charts as PNG files, making them easy to embed into reports or documentation. The little details mattered here, too—adjusting layouts with `tight_layout()`, for example, kept labels readable and the visuals tidy.

These graphics didn't just display data—they turned processing work into clear, interpretable stories, offering practical insights for anyone reviewing the project.

7. Conclusion

The Software/ML Technical Task Phase was a practical, challenging introduction to how software, data engineering, and analysis fit together. It was one long exercise in connecting good habits—like clean code, modular design, and version control—with the realities of wrangling, fixing, and interpreting messy data. If there was one core takeaway, it's that great analytics start long before the first chart: they begin with organizing, cleaning, and validating your data until the foundation is rock solid.

Working both by hand (manual parsing) and with advanced tools (Pandas) helped me appreciate both the fundamentals and the power of professional libraries. And as much as code matters, organizing the project—keeping functions clean, storing everything in version control, documenting as I went—made a world of difference.

Visualization wrapped the work in clarity, helping readers or collaborators see connections and statistics at a glance. That's what gives data its value: making truths discoverable and actionable, not hidden in tables of numbers.

Collectively, these tasks built a strong starting point for future projects in machine learning, software development, or data science. I came away not just with demystified Python and solid engineering skills, but with an appreciation for the full workflow—from raw data to actionable insights.

References

1. Python Software Foundation. Python 3 Documentation.
<https://docs.python.org/3/>

2. Pandas Development Team. Pandas Documentation.
<https://pandas.pydata.org/docs/>

3. Matplotlib Development Team. Matplotlib Documentation. **<https://matplotlib.org/stable/>**

4. Git Documentation. **<https://git-scm.com/docs>**

5. GitHub Documentation. **<https://docs.github.com/>**

Table 1. Summary of Tasks Completed

Task	Main Objective	Key Concept Learned
Task 1	Repository setup and Git basics	Version control, Git workflow, repository organization
Task 2	Python programming fundamentals	Functions, file handling, modular programming
Task 3	Manual CSV parser and Pandas comparison	CSV structure, type conversion, DataFrame operations
Task 4	Data cleaning and validation	Missing values, duplicates, standardization, validation
Task 5	Data visualization	Bar charts, scatter plots, graphical representation of data

Figure 1. Workflow of the Project

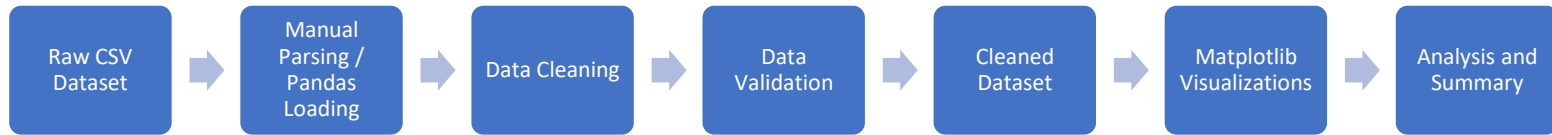


Table 2. Data Cleaning Rules

Data Issue	Cleaning Rule Applied
Duplicate rows	Removed duplicate records
Inconsistent domain names	Converted to standardized categories (ML, Web, Electronics, Mechanical)
Attendance values	Converted to numeric values and restricted to the valid range (0–100)
Score values	Converted number words to numeric values
Study hours	Converted textual values into numeric format
Height values	Converted all measurements to centimetres
Weight values	Converted all measurements to kilograms
Missing numeric values	Replaced using the median of the respective column
Submission status	Standardized to consistent "yes" and "no" values
Final dataset	Validated before generating outputs and visualizations